

AI Shopping Recommendation Chatbot

A Virtual Sales Associate for E-Commerce

Python 3.12

FastAPI

React + Vite

ChromaDB

Apify

OpenRouter LLM

Technical Documentation · System Design · Implementation Guide

Table of Contents

1.

Project Overview

Objectives, problem statement, and key features

2.

System Architecture

End-to-end pipeline and component design

3.

Technology Stack

Frontend, backend, AI/ML, and external APIs

4.

Implementation Workflow

Step-by-step data flow and code walkthrough

5.

Data Sources & Local Database

ChromaDB, JSON datasets, and categories

6.

Apify Integration

Online product scraping when local data is missing

7.

LLM Recommendation Engine

OpenRouter, scoring algorithm, and ranking

8.

Challenges & Solutions

Technical difficulties and how they were overcome

9.

Future Enhancements

Database integration, chat history, and roadmap

10.

Conclusion

Summary and project outcomes

1. Project Overview

The **AI Shopping Recommendation Chatbot** is an intelligent, context-aware virtual sales assistant designed to transform the e-commerce experience. Rather than forcing users to browse through thousands of products manually, this system understands natural language queries, extracts shopping intent, searches a local product database, and — when products are not found locally — automatically scrapes live data from major e-commerce platforms using **Apify**.

■ Core Objective

Build a conversational AI assistant that acts as a knowledgeable sales associate — understanding what a user wants, matching products to their intent, budget, and preferences, and presenting well-ranked, meaningful recommendations.

Key Features

- Natural language query understanding with intent detection
- Budget-aware product filtering and scoring
- Semantic vector search via ChromaDB embeddings
- Real-time product scraping via Apify when local data is absent
- Multi-session chat with context-aware follow-up handling
- Side-by-side product comparison with structured tables
- Bundle recommendation for complementary products
- LLM-generated conversational responses via OpenRouter
- Pagination support with 'Show More' functionality
- Gender-aware filtering and category detection

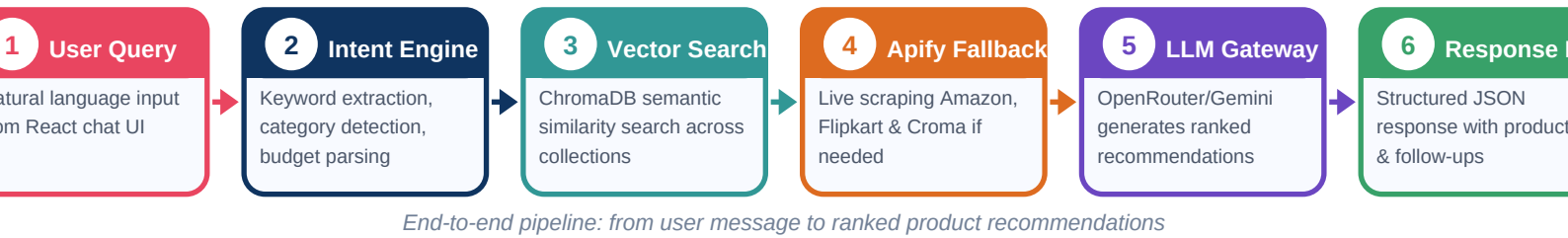
Problem Statement

Traditional e-commerce search engines rely on exact keyword matching, often failing users with casual, contextual, or intent-rich queries like *"I need something to wear to a friend's beach wedding under \$2000"*. This project solves that gap by combining:

- Large Language Model reasoning for deep query understanding
- Vector embeddings for semantic product matching beyond keywords
- Live web scraping as a fallback when the local catalog is incomplete
- Composite scoring that weighs price fit, ratings, and data quality simultaneously

2. System Architecture

The system follows a **layered pipeline architecture** with six distinct stages. Each stage is loosely coupled and can operate independently, enabling modular testing and future upgrades.



Architectural Layers

Layer	Component	Technology	Responsibility
Presentation	React Frontend	React 18 + Vite + Tailwind	Chat UI, product cards, session management
API Gateway	FastAPI Server	Python FastAPI + CORS	REST endpoints, streaming SSE, routing
Intelligence	Shopping Pipeline	LangChain-style pipeline	Query analysis, intent routing, orchestration
Data – Local	ChromaDB Collections	ChromaDB + Sentence Transformers	Semantic vector store, embedding search
Data – Live	Apify Scraper	Apify REST API	Real-time Amazon/Flipkart/Croma scraping
AI / LLM	LLM Gateway	OpenRouter / Gemini / Groq	Product explanations, comparisons, bundles
Scoring	Recommendation Service	Custom Python scoring	Composite ranking: similarity + price + rating

3. Technology Stack

Frontend
Framework: React 18 with Vite build tooling
Styling: Tailwind CSS for utility-first responsive design
State Mgmt: Zustand for lightweight global chat state
API Calls: Native fetch() with async/await
Chat UI: Custom ChatComponents.jsx with product cards, streaming indicators
Build Output: Optimized dist/ bundle deployed statically

Backend
Framework: FastAPI (Python) – async REST API with auto-docs
Runtime: Python 3.12 with uvicorn ASGI server
CORS: Open CORS middleware for local dev cross-origin requests
Validation: Pydantic schemas for all request/response models
Streaming: Server-Sent Events (SSE) for real-time chat stream
Entry Point: run.py → uvicorn backend.main:app --reload

AI / ML
Vector DB: ChromaDB – local persistent vector store with HNSW index
Embeddings: sentence-transformers (all-MiniLM-L6-v2) for semantic embeddings
LLM Gateway: OpenRouter API – routes to best available model
LLM Models: Fallback chain: OpenRouter → Gemini 1.5 Flash → Groq Llama3
NLP: Custom keyword extractor with regex + domain term bank
Scoring: Composite weighted scoring: similarity 40%, price 30%, rating 20%, completeness 10%

External APIs
Apify: E-commerce scraping tool – Amazon.in, Flipkart, Croma search pages
Tavily: Web search fallback for product discovery enrichment

OpenRouter: Unified LLM API gateway with model routing

Gemini: Google Generative AI API as primary LLM fallback

4. Implementation Workflow

The following describes the complete data flow from a user's message to the final recommendation response, tracing through every component in sequence.

Step 1 — User Sends Message

- User types a natural language query in the React chat interface (e.g., 'Best laptop under ₹80,000')
- The frontend sends a POST /chat request with the message, session ID, and chat history
- FastAPI validates the ChatRequest Pydantic schema and routes to chat_endpoint()

Step 2 — Intent & Keyword Extraction

- KeywordService.extract_detailed_intent() runs regex + domain term matching
- Extracts: category (e.g. 'laptops'), budget (80000), brand preferences, gender, style/occasion
- Determines response type: RECOMMEND / COMPARE / BUNDLE / FOLLOW_UP / GREETING / CLARIFICATION
- Generates search_queries list for downstream vector and web search

Step 3 — Local Vector Search (ChromaDB)

- VectorService.search_all_collections() queries ChromaDB across relevant collections
- Products are embedded via sentence-transformers and compared by cosine similarity
- Keyword-weighted re-scoring applied on top of semantic similarity scores
- Results deduped and filtered by budget, gender, and category constraints

Step 4 — Apify Fallback (if needed)

- If local results are < 3 or confidence is low, ApifyService.discover_products() is triggered
- Constructs Amazon.in, Flipkart, and Croma search URLs from extracted keywords
- Calls Apify e-commerce-scraping-tool actor via REST API (run-sync-get-dataset-items)
- Returns normalized product dicts matching internal schema; falls back to simulation if API fails

Step 5 — LLM Ranking & Response Generation

- RecommendationService.rank() applies composite scoring: semantic + price + rating + completeness
- LLMGateway.generate() sends context + top products to OpenRouter (or Gemini/Groq fallback)
- LLM generates a natural language explanation, comparison table, or bundle description
- Follow-up questions are auto-generated based on detected intent type

Step 6 — Response Serialization & Return

- ChatResponse Pydantic model assembled with: message, response_type, products, follow_ups
- Pagination token attached if more than 3 products available
- Background task queued to continue Apify discovery for session enrichment
- JSON response returned to frontend for rendering in chat UI with product cards

5. Data Sources & Local Database

The local product catalog is stored in **ChromaDB**, a persistent vector database. Products were pre-loaded from structured JSON datasets covering multiple e-commerce categories. During runtime, ChromaDB collections are queried semantically.

Local Dataset Categories

Category	File(s)	Product Types
Electronics	laptops.json, phones_data.json, smartphones.json	Laptops, smartphones, tablets
Fashion – Men	casual_wear.json, formal_wear.json, traditional_wear.json	Shirts, trousers, kurtas, formal suits
Fashion – Women	womens_clothing_data.json	Dresses, tops, ethnic wear
Kids	kids_clothing_data.json	Children's clothing
Footwear	shoes.json, shoes_data.json	Sneakers, formal, sandals, boots
Watches	watches.json, watches_data.json	Analog, digital, smartwatches
Sports	sports_wear.json, sports_and_outdoors.json	Activewear, gym gear, outdoor equipment
Home & Decor	home_decor.json, lighting_solutions.json, kitchen.json	Furniture, lighting, kitchenware
Toys	toys_and_games.json	Educational toys, board games, action figures
Cleaning	cleaning_supplies.json	Household cleaning products

ChromaDB Vector Store Details

Each product category maps to a separate ChromaDB collection. Products are embedded using **sentence-transformers/all-MiniLM-L6-v2**, producing 384-dimensional vectors. The HNSW index enables approximate nearest-neighbor search in milliseconds.

- Persistence: Local disk at backend/chroma_db/
- Embedding model: all-MiniLM-L6-v2 (384-dim)
- Distance metric: Cosine similarity
- Index type: HNSW (Hierarchical Navigable Small World graph)
- Top-K retrieval: Default n=10 per collection, re-ranked to top 3
- Collection naming: Matches product category (e.g. 'laptops', 'smartphones', 'fashion')

6. Apify Integration — Live Data Extraction

Apify is used as a live web scraping layer that activates when the local ChromaDB database does not contain sufficient matching products for a user's query. This ensures the chatbot always has relevant, current data — especially for niche queries, new product launches, or highly specific brand/model requests.

Why Apify?

Local datasets have finite coverage — they cannot include every product across all categories.

Apify's e-commerce-scraping-tool actor handles dynamic JavaScript-rendered pages on Amazon, Flipkart, and Croma.

It provides structured product data (name, price, ratings, images, specs) without maintaining custom scrapers.

Apify's cloud infrastructure handles anti-bot detection, IP rotation, and rate limiting automatically.

How Apify is Triggered

1. User sends a query → Intent extraction determines category and search keywords
2. Pipeline checks ChromaDB: if results < threshold or category not in local DB → Apify activates
3. `ApifyService.discover_products(intent)` generates search URLs for Amazon, Flipkart, and Croma
4. POST request sent to Apify REST API: `/v2/acts/apify~e-commerce-scraping-tool/run-sync-get-dataset-items`
5. Response (list of product dicts) normalized to internal schema via `_normalize_apify_items()`
6. Normalized products merged with local results and re-ranked by `RecommendationService`

Apify Actor Configuration

Parameter	Value	Description
Actor ID	apify~e-commerce-scraping-tool	Apify's official e-commerce multi-site scraper
listingUrls	Amazon + Flipkart + Croma URLs	Search result pages per keyword
maxProductResults	10 per URL	Cap per URL to balance speed and coverage
scrapeMode	AUTO	Automatic detection of page structure
Timeout	45 seconds	HTTP client timeout per API call
Fallback	<code>_simulate_discovery()</code>	Returns mock products if API unavailable

Data Normalization from Apify

Raw Apify output uses inconsistent field names across different e-commerce sites. The `_normalize_apify_items()` function maps raw fields to a consistent internal schema:

Internal Field	Apify Raw Fields (tried in order)	Fallback
name	name, title	Skipped if empty
price	offers.price, price, priceReal	None
mrp	offers.highPrice, mrp, listPrice, originalPrice	None

rating	aggregateRating.ratingValue, rating, stars	None
image_url	image, imageUrl, thumbnail	Empty string
product_url	url, productUrl, sourceUrl	Empty string
brand	brand.name, brand	Extracted from name via regex
specifications	specifications, specs	Empty dict
discount	discount, discountPercentage	Calculated from price/mrp

7. LLM Recommendation Engine

The recommendation engine combines deterministic scoring with LLM-powered natural language generation. Products are first ranked algorithmically, then explained conversationally by the language model.

Composite Scoring Algorithm

Every product receives a composite score between 0.0 and 1.0 based on four weighted factors:

Factor	Weight	Description
Semantic Similarity	40%	Cosine similarity of product embedding vs. query embedding from ChromaDB
Price Relevance	30%	How well price fits user budget. Exponential penalty for over-budget items. Category-specific lo
Product Rating	20%	Normalized star rating (rating / 5.0). Higher-rated products scored proportionally higher
Data Completeness	10%	Score based on presence of 7 key fields: name, price, image, rating, specs, brand, MRP

LLM Gateway & Model Chain

The system uses **OpenRouter** as a unified API gateway, with automatic fallback to alternative LLMs if the primary model is unavailable:

- **Primary:** OpenRouter API → routes to best available model (Claude, GPT-4, Mistral, etc.)
- **Fallback 1:** Google Gemini 1.5 Flash via GEMINI_API_KEY
- **Fallback 2:** Groq Llama-3 8B via GROQ_API_KEY (ultra-fast inference)
- **Simulation:** Template-based response if all LLM APIs fail

Response Types & LLM Usage

Response Type	Intent Trigger	LLM Role
RECOMMEND	Default product search	Explains top 3 products with reasoning
COMPARE	User says 'compare A and B'	Creates structured comparison table
BUNDLE	User asks for 'complete set'	Curates complementary product combo
FOLLOW_UP	References previous chat context	Maintains context, refines prior results
GREETING	Hello / General chat	Welcome message with example suggestions
NEEDS_CLARIFICATION	Ambiguous query	Asks targeted follow-up question
EXPLAIN	Asks about features/specs	Provides detailed product explanation

8. Challenges Faced & How They Were Solved

Challenge 1: Inconsistent Product Data from Apify

Problem: Different e-commerce sites (Amazon, Flipkart, Croma) use completely different JSON schemas. Field names like 'price' appear as 'offers.price', 'priceReal', 'currentPrice', or are nested inside 'offers' objects.

Solution: Built a comprehensive `_normalize_apify_items()` function that tries multiple field paths in priority order, with smart fallbacks (e.g. calculating discount from price/mrp if not provided, extracting brand via regex from product name). Logging all raw and normalized products to `logs/extracted_products.jsonl` for debugging.

Challenge 2: Gender Mis-filtering in Fashion Queries

Problem: When searching for 'women's kurta', the system would sometimes return men's products because text embeddings don't inherently distinguish gendered intent.

Solution: Implemented strict gender filtering in `RecommendationService.rank()`. `detect_gender()` uses regex patterns on the query to identify intent ('men' vs 'women'). Products are then checked via `_detect_product_gender()` which looks at the product's gender field and name/description. Mismatched gender products are removed before scoring.

Challenge 3: Budget-Aware Recommendations for Electronics

Problem: Budget relevance scoring was linear initially — a ₹5,000 laptop returned 'perfect price fit' for a ₹80,000 budget, which was misleading. Users want products close to their budget, not just under it.

Solution: Introduced category-specific price relevance logic in `_price_relevance()`. For premium categories (smartphones, laptops, appliances), products below 30% of budget score 0.15, while products at 70-100% of budget score 1.0. For fashion and accessories, any price under budget scores 1.0 (value perception is different).

Challenge 4: Slow Response for Live Scraping Queries

Problem: Apify scraping takes 30-45 seconds to return results. Making users wait that long for a chat response creates poor UX.

Solution: Implemented a two-phase response strategy: immediately return local ChromaDB results as Phase 1. Then trigger Apify discovery as a FastAPI BackgroundTask that runs asynchronously. Results from the background task are stored in product cache and delivered via 'Show More' pagination. A Server-Sent Events stream allows the frontend to receive updates when background data arrives.

Challenge 5: Multi-Turn Context in Follow-Up Queries

Problem: Users often ask follow-up questions like 'show me accessories for that' or 'what about cheaper options?' without repeating the original context.

Solution: The chat session maintains a history list passed with every request. The FOLLOW_UP intent type is detected when queries reference previous messages. The KeywordService augments the current query with keywords from the previous turn, and the LLM receives the full conversation history for contextual response generation.

Challenge 6: Embedding Large Product Datasets into ChromaDB

Problem: Embedding thousands of products with sentence-transformers was slow during initial setup, and ChromaDB would sometimes throw dimension mismatch errors when collections were re-initialized.

Solution: Built a dedicated `load_data.py` script that batch-processes products with error handling and progress tracking. Collections are versioned by category name, and existing collections are checked before re-embedding. The persistent `chroma_db/` directory eliminates re-embedding on restart.

9. Future Enhancements

The current implementation uses in-memory session state and file-based ChromaDB storage. Several enhancements are planned to scale the system, persist data reliably, and improve recommendation quality significantly.

Database Integration for Chat History

- ◆ Integrate PostgreSQL or MongoDB to persist all chat sessions and message history
- ◆ Store user preferences (preferred brands, budgets, past purchases) for personalization
- ◆ Enable cross-session recommendations: 'You previously looked at Nike shoes...'
- ◆ Build an admin dashboard to view conversation analytics, popular queries, and drop-off points
- ◆ Implement SQLAlchemy ORM layer on top of FastAPI for clean database access patterns

Enhanced Recommendation Quality

- ◆ Collaborative filtering: users with similar purchase history get similar recommendations
- ◆ Click-through tracking: products users click on boost their future recommendation scores
- ◆ Time-aware recommendations: seasonal products (winter wear, Diwali gifts) surface contextually
- ◆ Price drop alerts: notify users when a saved product's price drops below their budget
- ◆ Fine-tune sentence-transformer embeddings on e-commerce product domain data

Expanded Apify Data Sources

- ◆ Add Myntra, Nykaa, Meesho, Tata CLiQ as scraping targets for fashion and beauty
- ◆ Implement Apify product detail page scraper for complete specs and user review extraction
- ◆ Cache Apify results in Redis with TTL to avoid redundant scraping for popular queries
- ◆ Build a background cron job to refresh popular category data in ChromaDB nightly
- ◆ Use Tavily API for price comparison aggregation across multiple sellers

Frontend & UX Enhancements

- ◆ Voice input support using Web Speech API for hands-free shopping queries
- ◆ Product image search: upload a photo, find similar products semantically
- ◆ Wishlist and cart functionality with price tracking notifications

◆ A/B testing framework for different recommendation UI layouts
◆ PWA (Progressive Web App) with offline support and push notifications

Infrastructure & Scalability
◆ Docker containerization with docker-compose for one-command deployment
◆ Kubernetes-based horizontal scaling for the FastAPI backend
◆ Redis caching for frequent query results to reduce LLM API costs by 60-70%
◆ Replace local ChromaDB with Pinecone or Weaviate managed vector DB for production
◆ Implement rate limiting, API key authentication, and request logging middleware

10. Conclusion

The **AI Shopping Recommendation Chatbot** successfully demonstrates how modern AI techniques — large language models, vector embeddings, and real-time web scraping — can be combined to build a practical, scalable, and user-friendly virtual shopping assistant.

The project goes beyond simple keyword search to deliver a context-aware, intent-driven experience that understands natural language, respects user budgets, handles follow-up conversations, and dynamically sources fresh product data when the local catalog falls short.

Key Achievements

- Built a fully functional full-stack AI chatbot with React frontend and FastAPI backend
- Implemented semantic vector search using ChromaDB with 384-dim sentence-transformer embeddings
- Designed a composite scoring algorithm balancing relevance, price fit, ratings, and completeness
- Integrated Apify for live e-commerce scraping across Amazon, Flipkart, and Croma
- Achieved real-time response delivery with background task processing for slow data sources
- Handled complex NLP tasks: intent detection, budget extraction, gender filtering, follow-up resolution
- Created a multi-session chat architecture with per-session product caching and pagination
- Built fallback chains for all external dependencies (LLM → Gemini → Groq → Template)

Project Summary

This chatbot represents a production-ready prototype for an intelligent shopping assistant. The architecture is modular — each component (intent engine, vector search, Apify scraper, LLM gateway, scoring) can be upgraded independently. With the future enhancements outlined in Section 9, this system can scale to millions of products and thousands of concurrent users while maintaining the personalized, conversational experience that sets it apart from traditional e-commerce search.

<i>Backend</i>	Python + FastAPI	<i>Frontend</i>	React + Vite + Tailwind	<i>Vector DB</i>	ChromaDB + MiniLM	<i>Live Data</i>	Apify E-commerce Scraper	<i>LLM</i>	OpenRouter + Gemini + Groq
----------------	------------------	-----------------	-------------------------	------------------	-------------------	------------------	--------------------------	------------	----------------------------